FastenerERP Billing - Architecture & Scripts Report

1. Overview

FastenerERP Billing (internal name: `tanstack\_start\_ts`) is a full-stack billing and ERP solution built with a modern React-based tech stack. It supports both a web interface and a desktop application via Electron. The application revolves around authenticated user flows, customer management, invoice creation, and PDF generation.

2. Technology Stack

The project leverages a highly modern, type-safe stack:

- **Core Framework**: React 19, TypeScript
- **Meta-Framework & Routing**: TanStack Start & TanStack Router
- **Data Fetching & State**: TanStack Query (React Query) and Zustand (`src/lib/store.ts`)
- **Backend & Database**: Supabase (PostgreSQL, Auth)
- **Styling & UI**: Tailwind CSS v4, Radix UI, and shadcn/ui components
- **Desktop Wrapper**: Electron
- **PDF Generation**: `jspdf` and `html2canvas-pro`
- **Build Tool**: Vite (configured via `@lovable.dev/vite-tanstack-config`)

3. Core Architecture

Frontend Layer

- **Component-Driven**: UI is built using a rich set of Radix UI primitives and custom components (`src/components/ui`). Complex views like `InvoiceEditor.tsx` and `InvoicePdf.tsx` are modularized.
- **Routing**: Handled by TanStack Router. The `src/routes` directory defines the file-based routing structure. It includes a protected `\_authenticated` layout for routes like `/customers`, `/settings`, and `/invoices`.
- **State Management**: Zustand is used for global client state (`src/lib/store.ts`), while TanStack Query manages server state and caching.
- **Calculations & Logic**: Business logic, such as invoice calculations, is abstracted into utilities like `src/lib/calc.ts`.

Backend & Authentication

- **Supabase Integration**: The app uses Supabase for database interactions and authentication. The `src/integrations/supabase` folder contains auth middleware, client initialization for both server and client, and generated database types (`types.ts`).
- **Server-Side Rendering (SSR)**: Handled by TanStack Start and a custom server entry (`src/server.ts`).

Desktop (Electron) Integration

- The application can be packaged as a standalone Windows desktop app. The `electron/main.cjs` script bootstraps the Electron window, serving the Vite build output.

4. Directory Structure

```text
/
├── electron/            # Electron main process script (main.cjs)
├── src/                 # Frontend source code
│   ├── components/      # UI components (Radix/shadcn-ui), AppShell, InvoiceEditor
│   ├── hooks/           # Custom React hooks (e.g., use-mobile.tsx)
│   ├── integrations/    # 3rd-party integrations (Supabase clients, auth middleware, DB types)
│   ├── lib/             # Business logic, Zustand store, calc utilities, types
│   ├── routes/          # File-based routing (TanStack Router)
│   │   ├── _authenticated/  # Protected routes (customers, invoices, settings, index)
│   │   └── auth.tsx, reset-password.tsx
│   ├── router.tsx, server.ts # TanStack Router and Start configurations
│   └── styles.css       # Global Tailwind styling
├── supabase/            # Supabase configuration and database migrations
├── package.json         # Project dependencies and scripts
└── vite.config.ts       # Vite configuration (Lovable preset)
```

5. NPM Scripts

The `package.json` defines several scripts for development, building, and packaging:

Script	Command	Description
`dev`	`vite dev`	Starts the Vite development server.
`build`	`vite build`	Builds the production bundle for the web.
`build:dev`	`vite build --mode development`	Builds the app in development mode.
`preview`	`vite preview`	Serves the production build locally for preview.
`lint`	`eslint .`	Runs ESLint to check for code quality and errors.
`format`	`prettier --write .`	Formats the codebase using Prettier.
`electron`	`electron electron/main.cjs`	Starts the Electron desktop application (requires build first).
`electron:package:win`	`vite build \&\& @electron/packager . ...`	Builds the web app and packages it into a Windows `.exe` using `@electron/packager`.